# Beyond Majority Voting: Selecting LLM Answers via Hidden State Trajectory Probes

Nikolay Yudin
n.yudin@gmail.com

February 2026

**Abstract**

When a language model generates multiple candidate answers, how should we pick the best one? The default strategy—majority voting—treats the model as a black box, discarding everything except final answer strings. We show that the model's internal computations already contain a usable signal for answer quality, and that a remarkably simple method can extract it.

We propose *trajectory probes*: lightweight linear classifiers trained on hidden state features aggregated across the generation process. From each candidate answer, we extract mean, standard deviation, and final-token activations at eight evenly spaced layers, projected to 256 dimensions—a 6,144-dimensional trajectory fingerprint. A logistic regression probe trained with a pairwise ranking objective (RankNet) learns to prefer correct answers over incorrect ones from the same question.

On TriviaQA (Llama-3.1-8B-Instruct, $K$=4, $T$=0.3; mean±std over 3 seeds), the probe reaches 56.4%±3.9 versus 51.3%±3.9 for majority voting, recovering 58.4%±3.0% of the gap to the oracle upper bound, with a selection precision (PickAcc) of 91.2%±1.7% on questions where at least one sampled answer is correct. On MATH, gains are smaller and strongly $K$-dependent: at $K_{eval}$=2 the probe improves over majority voting by +2.1 points (3/3 seeds positive), while at the canonical $K_{eval}$=4 the improvement narrows to +0.6±1.0 points and is not statistically significant.

Two findings surprised us. First, the choice of training objective can matter more than standalone classifier quality: a binary classifier with higher cross-validated AUC can underperform a pairwise probe with lower AUC, because ranking among candidates is a different task than classifying correctness in isolation. Second, the per-layer signal distribution acts as a domain fingerprint—factual recall spreads information across layers while mathematical reasoning concentrates it in the final third—yet a single probe trained on mixed-domain data can match domain-specific specialists.

Our results suggest that the "verifier" for best-of-$K$ selection need not be a separate model or an additional LLM call. It can be a linear function of what the model already computes.[1]

## 1 Introduction

Sampling multiple outputs and selecting the best one is a standard trick for improving LLM reliability: self-consistency, pass@$k$, and simple majority voting (MV) are widely used baselines [9, 8, 3]. However, MV treats the model as a black box and ignores information the model computes *while* generating an answer. This paper asks a concrete question: *do hidden state trajectories*

[1]Code and reproducible notebooks are available at `https://github.com/nick-yudin/Generalization/tree/main/papers/Latent_control`. Development repo: `https://github.com/nick-yudin/grokking-research` (folder: `latent_configuration/02_latent_control`).

*contain a usable signal that distinguishes correct from incorrect candidates, strong enough to improve best-of-$K$ selection at essentially zero cost?*

We find that the answer is yes. We show that a single logistic regression probe, trained on aggregated hidden-state features, can consistently beat MV for answer selection. The probe does not require extra LLM calls, does not parse chain-of-thought text, and adds no latency at inference beyond a dot product.

**Contributions.**

- **Trajectory probes.** We define a simple representation of a generation attempt as a fixed-length *trajectory fingerprint* built from intermediate-layer activations aggregated over time.
- **Pairwise ranking objective.** We train a linear probe using a RankNet-style objective on (correct, incorrect) attempt pairs from the same question, directly optimizing the selection problem.
- **Empirical gains over MV.** On TriviaQA (3 seeds), trajectory probes improve selection accuracy by +5.1±0.1 points over MV at $K$=4. On MATH, gains are smaller at $K$=4 (+0.6±1.0 points) but are larger in the low-$K$ regime (+2.1 points at $K$=2).
- **Analysis.** We show objective mismatch effects (AUC $\neq$ selection quality) and characterize how signal localizes across layers differently for factual vs mathematical domains.

# 2 Problem Setup

Given a question $q$, we sample $K$ candidate answers $\{a_i\}_{i=1}^K$ from a base LLM under a fixed decoding policy (temperature $T$, max tokens, etc.). We then choose one answer to return. We compare four selectors:

- **Base:** return the first sampled answer ($a_1$).
- **Majority vote (MV):** choose the most frequent normalized answer string among $K$ samples.
- **Probe:** score each attempt using a learned trajectory probe and choose the highest-scoring attempt.
- **Oracle:** return a correct answer if any of the $K$ samples is correct (upper bound).

We also report two derived metrics:

- **PickAcc:** accuracy of the selector conditional on the oracle being able to pick a correct answer (i.e., the question has at least one correct attempt among $K$).
- **Recovery:** fraction of the oracle gap recovered: $\frac{\text{Acc(Probe)}-\text{Acc(Base)}}{\text{Acc(Oracle)}-\text{Acc(Base)}}$.

# 3 Method

## 3.1 Trajectory features

For each generated token step, we extract hidden states from a fixed subset of layers. We use 8 evenly spaced layers (`spread8`) from a 32-layer decoder. To keep the probe lightweight, we apply

Table 1: Main best-of-$K$ selection results for trajectory probes on TriviaQA and MATH. Base returns the first sample; MV is majority voting over $K$ samples; Probe selects the highest-scoring attempt; Oracle is the upper bound if any attempt is correct. PickAcc is conditional on Oracle being able to pick a correct answer; Recovery is the fraction of the oracle gap recovered. TriviaQA values are mean±std over 3 random seeds (500 test questions each; $T$=0.3, $K_{gen}$=4, canonical $K_{eval}$=4). MATH values are mean±std over 3 random seeds (500 test questions each; $T$=0.3, $K_{gen}$=6, canonical $K_{eval}$=4).

| Domain | $K_{gen}$ | Seeds | Base | MV | Probe | Δ(P–MV) | Oracle | PickAcc | Recovery |
|---|---|---|---|---|---|---|---|---|---|
| TriviaQA | 4 | 0/1/2 | 48.9%±4.0% | 51.3%±3.9% | 56.4%±3.9% | +5.1±0.1 pp | 61.8%±3.1% | 91.2%±1.7% | 58.4%±3.0% |
| MATH | 6 | 0/1/2 | 41.7%±0.8% | 47.9%±0.5% | 48.5%±1.5% | +0.6±1.0 pp | 58.3%±1.0% | 83.1%±1.9% | 40.8%±6.0% |

a deterministic Gaussian random projection (4096→256) to each layer's hidden state. For each attempt, we aggregate the projected per-token activations across time using three statistics:

$$\phi(a) \; = \; [\; \mu \;\oplus\; \sigma \;\oplus\; h_{\text{last}} \;] \in \mathbb{R}^{6144},$$

where $\mu$ and $\sigma$ are mean and standard deviation over the generation trajectory, and $h_{\text{last}}$ is the final-token activation. We refer to $\phi(a)$ as the attempt's *trajectory fingerprint*.

### 3.2 Pairwise ranking probe

Binary correctness classification is not the selection problem: we do not need calibrated correctness probabilities, we need a *ranking* within the $K$ candidates for the same question. We therefore train a probe using pairwise differences between a correct attempt and an incorrect attempt from the same question:

$$x = \phi(a^+) - \phi(a^-), \quad y = 1; \qquad x' = \phi(a^-) - \phi(a^+), \quad y' = 0.$$

We fit a logistic regression model $s(x) = w^\top x + b$ with standard cross-entropy loss over these pairs, equivalent to a RankNet-style objective [2]. At inference, we score each attempt $a_i$ by $w^\top \phi(a_i)$ and select the maximum.

## 4 Experiments

### 4.1 Setup

We evaluate on two domains: (i) TriviaQA (factual short-answer QA) [6] and (ii) MATH (mathematical reasoning with boxed final answers) [4]. Our base model is `meta-llama/Meta-Llama-3.1-8B-Instruct`. We generate candidates with temperature $T$=0.3. Canonical configurations are described in the release manifest (see footnote in the abstract).

### 4.2 Main results

## 5 Analysis

### 5.1 Per-layer signal as a domain fingerprint

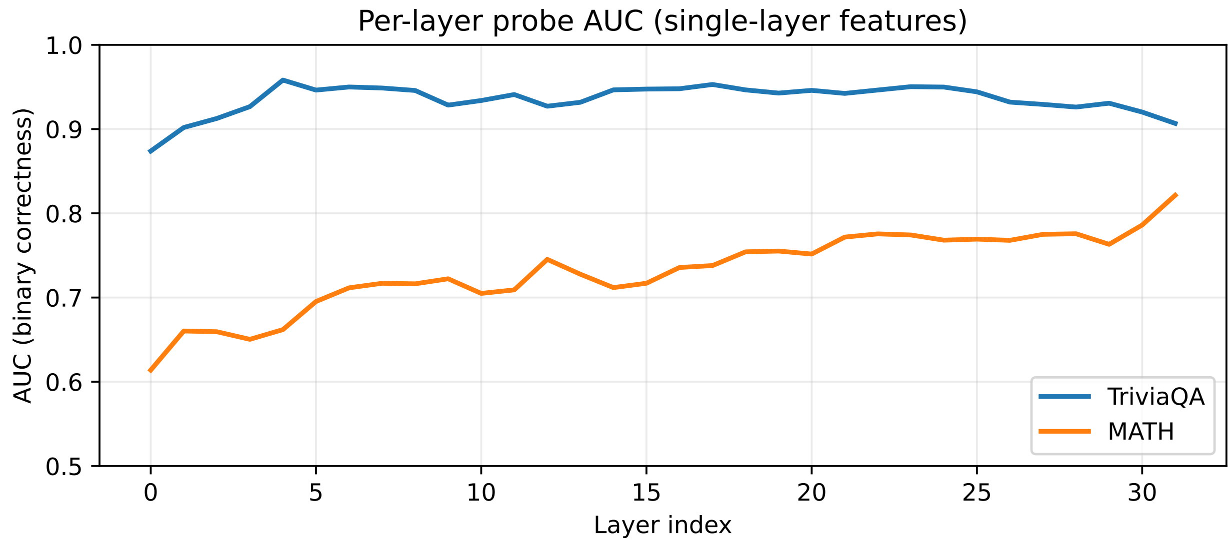
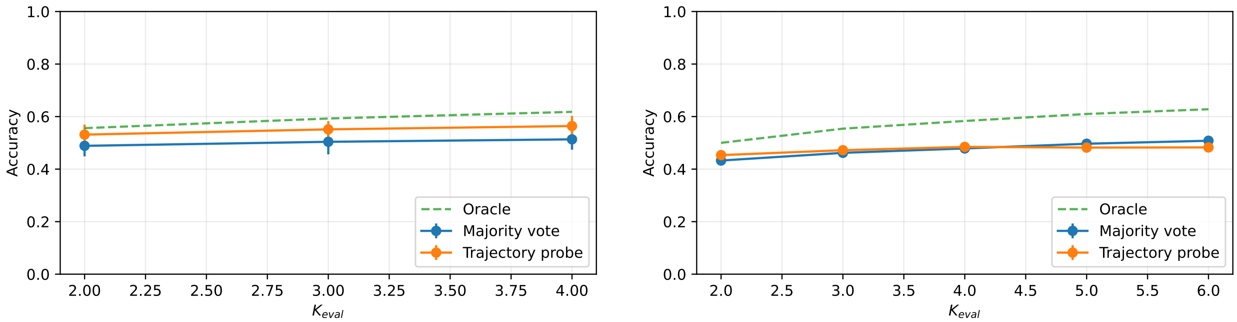



Figure 1: Per-layer AUC of a binary probe trained on single-layer features (TriviaQA vs MATH). Factual recall shows broadly distributed signal; MATH concentrates in late layers.

### 5.2 Objective mismatch: why AUC can mislead

In a MATH ablation with fixed features and $K_{train}$=$K_{eval}$=4, a binary classifier attains higher cross-validated AUC than the pairwise probe, yet fails to beat MV in selection accuracy, while the pairwise probe improves selection. This illustrates that evaluating a probe as an isolated classifier can be misaligned with the *within-question* selection objective.

## 5.3 K-dependence




(a) TriviaQA (b) MATH

Figure 2: Selection accuracy vs $K_{\text{eval}}$ for majority voting and trajectory probes (mean±std over seeds). TriviaQA shows consistent probe gains across $K$; MATH shows a low-$K$ advantage that vanishes and reverses at higher $K$, where majority voting benefits more from additional samples.

Figure 2 reveals a qualitative difference between domains. On TriviaQA, the probe improves over MV across the evaluated range of $K$ and the gap remains stable. On MATH, the probe has its clearest advantage in the low-$K$ regime ($K$=2–4), but MV overtakes at higher $K$. A plausible interpretation is that MV benefits from additional samples when the answer distribution has a coherent majority mass, while the probe's ranking signal may become less reliable as candidate diversity increases and the "best" attempt is harder to separate from near-misses.

## 5.4 Length confound

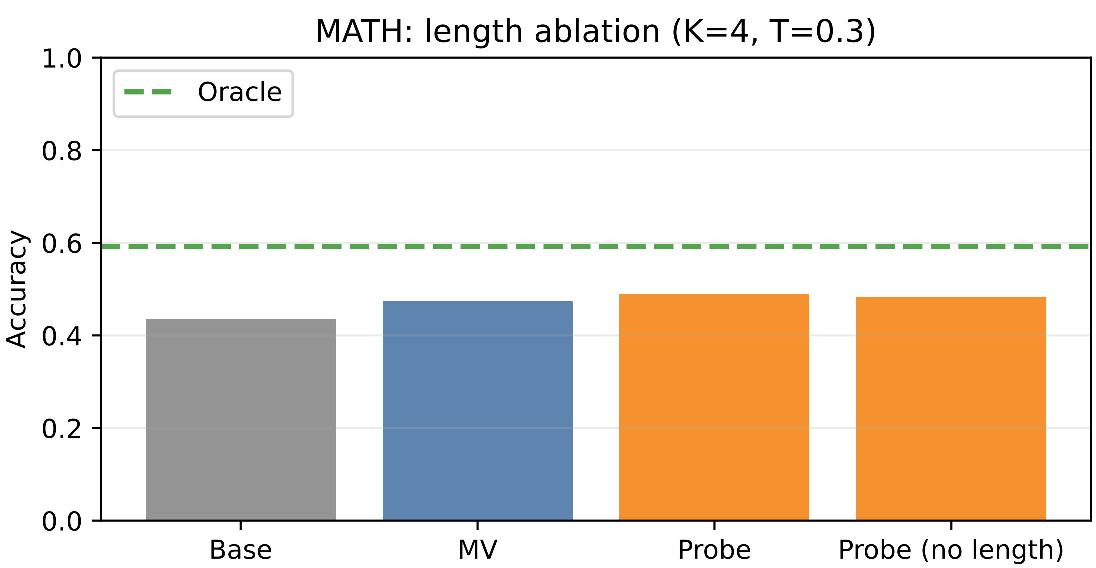



Figure 3: MATH length ablation: removing token-count features reduces probe performance slightly but preserves gains over MV, suggesting the probe captures more than answer length.

Because MATH solutions vary widely in length, an obvious failure mode is that the probe might learn a cheap heuristic (e.g., “longer explanations are more likely correct”). We test this by removing length-related features from the probe input. As shown in Figure 3, performance drops slightly, but the probe remains competitive with MV, indicating the trajectory fingerprint contains quality information beyond superficial length correlates.

### 5.5 Cross-domain transfer

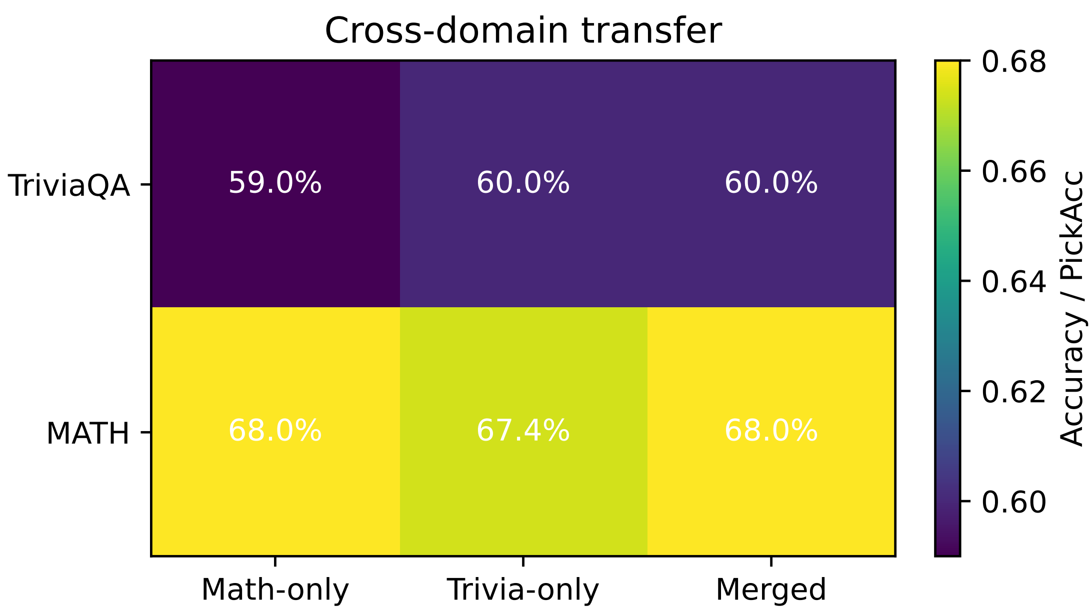



Figure 4: Cross-domain transfer: a merged probe can match domain-specific probes, despite low cosine similarity between learned weights.

Finally, we ask whether the trajectory signal is domain-specific. In a transfer study, probes trained on one domain degrade when evaluated on the other, but a single merged probe trained on mixed-domain data matches the domain-specific specialists. This suggests that “universal” trajectory probes may be possible without strong negative transfer, even if the underlying linear directions used by each domain are nearly orthogonal.

## 6 Related Work

Selecting among multiple samples is closely related to self-consistency and pass@$k$ evaluation [8, 3]. Our work is also adjacent to learned verifiers and reward models used for selection or reranking, which typically require additional model calls or training a separate evaluator [7]. Finally, trajectory probes build on a large body of work probing internal representations with simple classifiers [5, 1].

## 7 Limitations

Trajectory probes require access to internal activations, which may not be available via hosted APIs. The method selects among existing candidates and does not generate new answers. We evaluate one model family and a limited set of domains; generalization to other models and tasks remains to be explored. Finally, gains on MATH are modest at the canonical $K$=4 setting and appear sensitive to $K$, suggesting that best-of-$K$ selection may require domain- and regime-specific objectives.

## 8 Conclusion

Majority voting discards most of what a model computes. Trajectory probes show that a lightweight linear readout of hidden-state dynamics can serve as a cheap selector for best-of-$K$ answers, improving accuracy with negligible inference overhead. In future work, we aim to turn this selection signal into an actionable control loop: dynamically allocating computation, retries, or alternative decoding based on latent trajectory feedback.

## References


[1] Yonatan Belinkov and James Glass. Analysis methods in neural language processing: A survey. *Transactions of the Association for Computational Linguistics*, 7:49–72, 2019.

[2] Christopher Burges, Tal Shaked, Erin Renshaw, Ari Lazier, Matt Deeds, Nicole Hamilton, and Greg Hullender. Learning to rank using gradient descent. In *Proceedings of the 22nd International Conference on Machine Learning (ICML)*, 2005.

[3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[4] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. *arXiv preprint arXiv:2103.03874*, 2021.

[5] John Hewitt and Percy Liang. Designing and interpreting probes with control tasks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019.

[6] Mandar Joshi, Eunsol Choi, Daniel Weld, and Luke Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. *arXiv preprint arXiv:1705.03551*, 2017.

[7] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *arXiv preprint arXiv:2203.02155*, 2022.

[8] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.

[9] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *arXiv preprint arXiv:2201.11903*, 2022.